

Reliable Non-Leveled Homomorphic Encryption for Web Services

Baigang Chen*

University of Minnesota
Twin Cities, MN, United States
chen9464@umn.edu

Dongfang Zhao

University of Washington
Tacoma, WA, United States
dzhao@cs.washington.edu

Abstract

With the ubiquitous deployment of web services, ensuring data confidentiality has become a challenging imperative. Fully Homomorphic Encryption (FHE) presents a powerful solution for processing encrypted data; however, its widespread adoption is severely constrained by two fundamental bottlenecks: substantial computational overhead and the absence of a built-in automatic error correction mechanism. These limitations render the deployment of FHE in real-world, complex network environments impractical.

To address this dual challenge, this work puts forward a new FHE framework that enhances computational efficiency and integrates an automatic error correction capability through new encoding techniques and an algebraic reliability layer. Our system was validated across several web workloads, including encrypted inference on MNIST and CIFAR-10, federated aggregation with non-IID data, and streaming analytics on household power consumption data. Experimental results demonstrate significant performance improvements, particularly with large polynomial modulus degrees such as $N = 8192$, while maintaining task accuracy within 0.5% of the plaintext baseline. Furthermore, our error correction mechanism reduces the service failure rate to below 0.5% even under harsh, bursty network fault conditions.

CCS Concepts

• Security and privacy;

Keywords

Privacy-Preserving Machine learning, Fully Homomorphic Encryption

ACM Reference Format:

Baigang Chen and Dongfang Zhao. 2026. Reliable Non-Leveled Homomorphic Encryption for Web Services. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792476>

1 Introduction

Ensuring data privacy is a central challenge in modern web services, particularly for data-intensive applications like machine learning [27]. Fully Homomorphic Encryption (FHE) provides a powerful cryptographic foundation for such services. The CKKS scheme [14], which supports arithmetic on encrypted real numbers, is especially

well-suited for machine learning workloads. However, two significant obstacles hinder the practical and dependable deployment of CKKS in real-world services. The first is the rigidity of the conventional leveled execution paradigm, which requires developers to predict the computation's multiplicative depth in advance. The second is the fragility of ciphertexts, which can be corrupted during transport across distributed system components, degrading accuracy and reliability.

These challenges manifest as concrete engineering and performance problems. In a standard leveled CKKS workflow, noise from multiplications is managed by descending a pre-selected chain of moduli via a Rescale operation. This forces a trade-off: developers must either overestimate the required multiplicative depth, leading to inefficiently large parameters, or risk computation failure if the depth is underestimated. This upfront depth-planning requirement complicates development and can lead to unnecessary latency. Concurrently, in realistic deployment settings where ciphertexts traverse clients, servers, and storage systems, even infrequent bit-flips from network or hardware faults can cause complete decryption failures or silent numerical errors [31]. Conventional integrity mechanisms like hash-based retries are often unsuitable, as the additional round-trips they require can substantially inflate the tail latency of the service [41].

This paper presents a new FHE framework designed to address these dual challenges of usability and reliability. Our first component is a non-leveled variant of CKKS that operates at a fixed modulus. It replaces the standard Rescale operation with a lightweight Refresh operation, which periodically resets the ciphertext's scale to maintain numerical precision. This approach removes the need to plan computational depth, simplifying parameter selection and service deployment. Our second component is a lightweight error-correction layer integrated with a permutation-based interleaver. This mechanism effectively disperses bursty transmission errors across the ciphertext structure, enabling their correction with only millisecond-level overhead while remaining fully compatible with homomorphic operations.

We implemented our design in HELIB and validated it on three common web service workloads: encrypted inference, federated aggregation, and streaming analytics. The evaluation shows that our system achieves end-to-end latency comparable to or better than standard leveled CKKS, especially at larger parameters like $N = 8192$. It demonstrates strong fault tolerance under bursty channel conditions, maintaining task accuracy within 0.5 percentage points of plaintext baselines where traditional FHE would fail.

Contributions:

1. We present a fixed-modulus CKKS workflow that uses a periodic Refresh operation in place of leveled Rescale. This eliminates the need for multiplicative depth planning, simplifying the parameterization and deployment of FHE-based web services.

*Most of this work was carried out at the University of Washington, Seattle.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792476>

2. We design a code-based reliability layer combined with a permutation interleaver. This layer disperses and corrects burst errors that occur during data transmission with low runtime overhead, enhancing the robustness of FHE in web services.
3. Our HELIB implementation, evaluated on three web workloads, matches or improves on leveled CKKS latency for common polynomial modulus degrees of 4096 and 8192. For a millisecond-level overhead, its reliability layer cuts the failure rate to below 0.5% under burst faults, while keeping accuracy within 0.5 percentage points of the plaintext baseline.

2 Related Work

In this section, we review the developing history of FHE and the current works that bridge the gap between theoretical foundations and real-life deployments.

2.1 Fully Homomorphic Encryption

Early fully homomorphic encryption (FHE) enabled exact computation over modular rings with noise controlled by modulus switching or bootstrapping. Gentry's construction [22] led to leveled schemes such as BGV [8], BFV [19], and DGHV [45]. TFHE [16] advanced bit-level computation with fast gate bootstrapping. Recent surveys provide implementation guidance and trade-offs for practical deployments [23, 50, 51]. These designs are strong for exact integer logic but map poorly to the real-valued kernels that dominate privacy-preserving machine learning (PPML) on the web.

CKKS [14] addresses PPML by supporting approximate arithmetic and SIMD packing, enabling linear algebra over encrypted feature vectors and model weights. CKKS underpins many PPML pipelines in inference and analytics, yet three issues persist in practice: rapid noise growth that forces careful scale management, rescaling chains that complicate parameter tuning, and bootstrapping costs that limit depth and throughput. In distributed settings, ciphertext transmission adds another failure mode: rare bit flips can derail decryption and downstream ML tasks. Our work targets these PPML pain points by removing the leveled schedule and adding an algebraic reliability layer that fits the adjusted CKKS ring, improving robustness while preserving the programming model.

2.2 FHE System Optimization

Recent efforts have been focusing on improving the practical usability and performance of CKKS through software libraries and hardware acceleration. Libraries such as Microsoft SEAL [20], PALISADE [42], and Lattigo [37] offer high-level CKKS APIs alongside optimized FFT back-ends, while GPU-accelerated solutions [17, 38, 46, 52] provide deployment strategies tailored for commercial cloud platforms. Advances in CKKS optimization in low-complexity ciphertext multiplication [1, 15] and client-efficient lightweight variants [13] further reduce the theoretical complexity. Concurrently, coded-BKW techniques, combining the Blum–Kalai–Wasserman algorithm with algebraic error-correcting codes, have been investigated to improve decryption robustness [32]. However, beyond those approaches, system-level implementations [33] that focus on efficient computation still involve significant parameter inflation and remain limited to integer-based FHE schemes. Our work presents a unified BCH-coded framework based on the CKKS

scheme that supports approximate-number arithmetic, achieves strictly bounded multiplicative noise growth, and guarantees deterministic, bit-exact decryption without compromising IND-CPA security.

Moreover, to address storage and bandwidth limitations in practical deployments, several advanced compression and key management techniques have been proposed. Additive-HE helper compression reduces ciphertext sizes in LWE-based schemes by exploiting homomorphic aggregation [30]. Parallel-caching frameworks such as PawCache [44] and SILCA [49] precompute rotation and facilitate relinearization keys in radix form, significantly reducing key-generation latency and facilitating efficient batch evaluation. These system-level optimizations represent promising directions for integration into the non-leveled CKKS framework to enhance its performance and deployability.

2.3 FHE Bootstrapping and Fault Tolerance

On the bootstrapping front, polynomial-switching techniques with multi-digit decomposition achieve sub-second refresh times [35], while Bit-wise and small-integer bootstrapping variants [4, 5, 43] have demonstrated the feasibility of exact arithmetic over small domains at competitive speeds. Bootstrapping for binary input and non-sparse keys are efficient with modulus raising techniques [7, 11, 13]. More recent parallelized bootstrapping approaches [12] leverage multi-core and SIMD architectures to further reduce latency. At the protocol level, CKKS has been adopted in privacy-preserving analytics, secure multi-party computation, and end-to-end encrypted machine learning pipelines [25]; its IND-CPA security model has been rigorously formalized and extended to withstand adaptive adversaries [26].

In the quantum domain, early fault-tolerant FHE constructions employ CSS stabilizer codes to preserve logical qubit integrity during arbitrary quantum computations. Liang and Yang's QFHE scheme [29] uses transversal Clifford gates and magic-state injection to maintain fault tolerance under encryption, while later refinements unify encoding and encryption within a single CSS layer, thereby improving both security levels and correctable-fault threshold [40].

To the best of our knowledge, no prior work combines the approximate-arithmetic capabilities of CKKS with an algebraic outer error-correcting code that remains closed under the scheme's ring operations. We close this gap by (i) proposing a *non-leveled* CKKS variant that operates at a fixed scale; and (ii) integrating a Hensel-lifted BCH ideal, together with automorphism-based interleaving, so that codewords are preserved under addition and multiplication. This composition enables deterministic, bit-exact recovery without degrading IND-CPA security.

3 Methodology

In this section, we describe our core construction and analysis.

3.1 Building Blocks

All logarithms are base 2 unless stated otherwise. For a real number r , let $\lceil r \rceil$ be the nearest integer, rounding up in case of a tie. For an integer q , we identify $\mathbb{Z}_q$ with $\mathbb{Z} \cap (-q/2, q/2]$, and write $[z]_q$ for the reduction of z into that interval.

3.1.1 Cyclotomic Ring and Canonical Embedding. Let $M \in \mathbb{N}$ and $\Phi_M(X)$ be the M -th cyclotomic polynomial of degree $N = \varphi(M)$. We work over $R = \mathbb{Z}[X]/(\Phi_M)$ and its residue ring $R_q = R/qR$. Write $a \in R$ as $a(X) = \sum_{j=0}^{N-1} a_j X^j$ and identify it with its coefficient vector $(a_0, \dots, a_{N-1}) \in \mathbb{R}^N$; we use the usual coefficient $\|\cdot\|_\infty$ and $\|\cdot\|_1$ norms. The canonical embedding $\sigma(a)$ evaluates a at the primitive M -th roots of unity: $\sigma(a) = (a(\zeta_M^j))_{j \in \mathbb{Z}_M^*} \in \mathbb{C}^N$ with $\zeta_M = e^{-2\pi i/M}$, and we set $\|a\|_{\text{can}, \infty} := \|\sigma(a)\|_\infty$; note $\|\sigma\|_{\text{op}} \leq \sqrt{N}$. The image of σ lies in the real Hermitian subspace $\{z \in \mathbb{C}^N : z_{-j} = \bar{z}_j\}$, which (via a unitary change of basis) is isometric to $\mathbb{R}^N$ [39].

3.1.2 Gaussian Sampling and RLWE. Let H be the Hermitian space from above. For $r > 0$, write the (spherical) Gaussian on H as $\mathcal{D}_H(0, r^2 I)$ with density proportional to $\exp(-\pi \|z\|^2 / r^2)$. For an elliptical variant with per-coordinate scales $r_1, \dots, r_N > 0$, use $\mathcal{D}_H(0, \text{diag}(r_1^2, \dots, r_N^2))$. If $U : \mathbb{R}^N \rightarrow H$ is a fixed real isometry, then sampling $z \leftarrow \mathcal{N}(0, \text{diag}(r_1^2, \dots, r_N^2))$ and outputting Uz realizes this distribution. Mapping via $\text{CRT}_M^{-1} \circ U$ gives a continuous distribution Ψ_r over $R \otimes \mathbb{R} \cong \mathbb{R}[X]/(\Phi_M)$. Discretizing to the dual lattice by rounding yields the discrete Gaussian $\chi := \lfloor \Psi_r \rfloor_{R^\vee}$, which we use as the error distribution.

Definition 1 (RLWE distribution). Let $q \geq 2$, $R_q := R/qR$, and $R_q^\vee := R^\vee/qR^\vee$. For a secret $s \in R_q^\vee$ and error χ , define $A_{q,\chi}(s)$ over $R_q \times R_q^\vee$ by: sample $a \leftarrow R_q$ uniformly and $e \leftarrow \chi$, then return $(a, a \cdot s + e)$.

Definition 2 (Decisional RLWE). Given oracle access to i.i.d. samples, distinguish whether they come from $A_{q,\chi}(s)$ for a secret s drawn from a fixed distribution D over $R^\vee \pmod{q}$ or from the uniform distribution on $R_q \times R_q^\vee$.

3.1.3 BCH and Hensel Lifting. Let $n = 2^m - 1$ and $\alpha \in \mathbb{F}_{2^m}$ be primitive. For designed distance $\delta \geq 2t + 1$ and a consecutive index set $T = \{b, \dots, b + \delta - 2\} \pmod{n}$, the binary BCH code $\text{BCH}(n, k, t)$ is the cyclic code generated by $g_2(X) = \text{lcm}\{M_i(X) : i \in T\}$, where M_i is the minimal polynomial of α^i over $\mathbb{F}_2$. Then $k = n - \deg g_2$ and $d_{\min} \geq \delta$ [6]. A systematic encoder maps a message $u(X)$ to $c(X) = u(X)X^{n-k} + (u(X)X^{n-k} \bmod g_2(X))$. Algebraic decoding uses syndromes, Berlekamp–Massey (or EEA), and Chien search. For odd N , over $\mathbb{F}_2$ we have $X^N + 1 \equiv X^N - 1$, so we may choose a binary BCH generator $g_2 \mid (X^N + 1)$ from a consecutive root set modulo N . Since $\gcd(g_2, (X^N + 1)/g_2) = 1$ in $\mathbb{F}_2[X]$, Hensel's lemma lifts g_2 uniquely to a monic $g_k \in \mathbb{Z}_{2^k}[X]$ with $g_k \mid (X^N + 1)$ and $g_k \equiv g_2 \pmod{2}$.

3.2 Non-leveled FHE Construction

In this subsection, we introduce a *non-leveled* CKKS-based scheme that preserves the standard plaintext/ciphertext structure and programming model, while replacing leveled modulus switching with a lightweight REFRESH and fixed-scale management. We retain the canonical embedding, RLWE-based encryption, and SIMD packing; the only structural change is to eliminate Rescale and instead invoke a periodic REFRESH, a homomorphic re-encryption with Gaussian flooding, that maintains a single modulus and a fixed scale Δ . We first present the high-level workflow and then a concrete instantiation of this variant.

Our analysis shows that, without a modulus chain, the scheme controls noise growth under an explicit threshold, achieves quasi-linear complexity in the ring degree N (via NTT-based polynomial arithmetic), incurs only modest memory overhead, and supports homomorphic evaluation of general analytic functions.

3.2.1 Framework Construction. Similar to CKKS, our variant of CKKS's decryption follows the simple form $\langle c, \text{sk} \rangle = m + e$, where e is a small noise term. To ensure security, the encryption operation intentionally introduces a controllable small noise. During homomorphic computations, this error is monitored and refreshed when it exceeds the threshold to prevent error accumulation. While CKKS employs a rescaling mechanism to discard imprecise least significant bits (LSBs) and manage noise, our framework instead relies on an elementary bootstrapping to suppress noise growth.

Algorithm 1 is a concrete construction of our framework based on CKKS tailored for approximate computation over Gaussian integers with bounded-coefficient polynomial encoding and decoding. Correctness and some essential lemmas for error bounding are in the appendix.

Recall in standard CKKS [14, Section. 3.4 Lemma 3],

$$B_{\text{mult}}^{\text{std}} = v_1 B_2 + v_2 B_1 + B_1 B_2 + P^{-1} q_\ell \delta \sigma N / \sqrt{3} + N + 8/3 \sqrt{h}.$$

The terms v_1 and v_2 represent rescaling factors applied to the plaintext values encoded in the two input ciphertexts, while B_1 and B_2 denote the corresponding upper bounds on their noise magnitudes. The parameter P refers to the modulus employed during rescaling in standard CKKS, and q_ℓ is the modulus at level ℓ in the modulus chain. Given Lemma 3, we can compare the noise growth behavior of standard CKKS with our variant. Given $(c_1, B_1), (c_2, B_2)$, in our CKKS variant (eliminate Δ for a fair compare):

$$B_{\text{mult}}^{\text{var}} = v(B_2 + B_1) + B_1 B_2 + 16\sqrt{3}\sigma^2 N.$$

To determine when the noise resulting from our CKKS variant's multiplication operation is smaller than that of the standard CKKS, we have the following Proposition 1.

PROPOSITION 1. *Let c_1 and c_2 be fresh ciphertexts in both schemes with identical $B_1 = B_2 = B_{\text{enc}}$, $v_1 = v_2 = \Delta$, $B = \frac{\Delta}{2}$, and h is the Hamming weight from key sampling. Denote the worst-case bound returned by the respective lemmas as $B_{\text{mult}}^{\text{std}}$ and $B_{\text{mult}}^{\text{var}}$. We have the following:*

$$16\sqrt{3}\sigma^2 < \frac{8q_\ell}{\sqrt{3}P} + \frac{8\sqrt{h}}{\sqrt{3}\sqrt{N}} + 1 \implies B_{\text{mult}}^{\text{var}} < B_{\text{mult}}^{\text{std}}.$$

Proof in Appendix C.7. $\square$

This inequality holds trivially under the setting of CKKS, where q_ℓ is sufficiently large, proving the reduction of noise magnitude of the proposed non-level CKKS scheme. Lastly, let c be an encryption of a message m with noise at most B . Then $\text{Refresh}(\text{pk}, c)$ outputs a ciphertext c' that is computationally indistinguishable from a fresh encryption of a rounded message m' , with noise at most $B_r = \alpha B + \beta$, where α, β determined by the initial parameters as shown in Appendix B.

3.2.2 Support for Analytic Functions. We present algorithms for evaluating common circuit components in practical applications and analyze the associated error growth under the construction

Algorithm 1 Concrete Framework Construction

```

1: Procedure KEYGEN( $1^\lambda$ )
2:  $M \leftarrow M(\lambda)$ ,  $h \leftarrow h(\lambda)$ ,  $P \leftarrow P(\lambda)$ ,  $\sigma \leftarrow \sigma(\lambda)$ 
3:  $\tau \leftarrow 2^K \cdot B_{\max}$  for some  $K$ 
4: Sample  $s \leftarrow \text{HWT}(h)$ ,  $a \leftarrow R$ ,  $e \leftarrow \text{DG}(\sigma^2)$ 
5:  $\text{sk} \leftarrow (1, s)$ ;  $b \leftarrow -a \cdot s + e$ ;  $\text{pk} \leftarrow (b, a)$ 
6: Relinearization key:  $a_0 \leftarrow R$ ,  $e_0 \leftarrow \text{DG}(\sigma^2)$ ,  $b_0 \leftarrow -a_0 s + e_0 + s^2$ ;
    $\text{evk} \leftarrow (b_0, a_0)$ 
7: Boot key:  $\text{bk} \leftarrow \text{ENC}(\text{pk}, s)$ 
8: return ( $\text{sk}, \text{pk}, \text{evk}, \text{bk}, \tau$ )

9: Procedure ENCODE( $z; \Delta$ )
Require:  $z = (z_j)_{j \in T} \in \mathbb{Z}[i]^{N/2}$ 
10:  $c \leftarrow \lfloor \Delta \pi^{-1}(z) \rfloor_{\sigma(R)}$ 
11: Apply inverse canonical embedding to  $c$  to obtain  $m(X)$ 
12: return  $m(X)$ 

13: Procedure DECODE( $m(X); \Delta$ )
14:  $z_j \leftarrow \lfloor \Delta^{-1} m(\zeta_M^j) \rfloor$  for  $j \in T$ 
15: return  $z = (z_j)_{j \in T}$ 

16: Procedure ENC( $m(X)$ )
17: Sample  $v \leftarrow \text{ZO}(0.5)$ ,  $e_0, e_1 \leftarrow \text{DG}(\sigma^2)$ 
18: return  $v \cdot \text{pk} + (m(X) + e_0, e_1)$ 

19: Procedure DEC( $c = (b, a)$ )
20: return  $b + a \cdot s$ 

21: Procedure ADD( $((c_1, \Delta_1), (c_2, \Delta_2))$ ) with  $\Delta_1 \leq \Delta_2$ 
22:  $c_{\text{add}} \leftarrow \text{MULTCONST}(\frac{\Delta_2}{\Delta_1}, c_1) + c_2$ ; set  $\Delta_1 \rightarrow \Delta_2$ 
23: return  $\text{MULTCONST}(\frac{\Delta}{\Delta_2}, c_{\text{add}})$ 

24: Procedure MULT( $c_1 = (b_1, a_1)$ ,  $c_2 = (b_2, a_2)$ )
25:  $(d_0, d_1, d_2) \leftarrow (b_1 b_2, a_1 b_2 + a_2 b_1, a_1 a_2)$ 
26:  $c_{\text{mult}} \leftarrow (d_0, d_1) + d_2 \cdot \text{evk}$ ;  $\Delta_{\text{mult}} \rightarrow \Delta_1 \Delta_2$ 
27: return  $\text{MULTCONST}(\frac{\Delta}{\Delta_1 \Delta_2}, c_{\text{mult}})$ 

28: Procedure THRESH( $B_{\max}, B_0$ )
29: return ( $B_0 > B_{\max}$ )

30: Procedure REFRESH( $c = (c_0, c_1)$ ,  $\text{bk}$ ,  $\Delta$ ,  $\tau$ )
31: Homomorphic decrypt: obtain  $\text{Enc}(m+e)$ 
32:  $t \leftarrow \text{MULTCONST}(c_1, \text{bk})$  (equals  $\text{Enc}(c_1 \cdot s)$ )
33:  $u \leftarrow \text{ADDCONST}(t, c_0)$  ( $u = \text{Enc}(m+e)$ )
34: Approx. rounding: choose  $R$  with  $R(x) \approx \text{round}(x/\Delta)$ 
35:  $w \leftarrow \text{EVALPOLY}(u, R)$  ( $w \approx \text{Enc}(\text{round}((m+e)/\Delta))$ )
36: Re-encode to target scale
37:  $c_{\text{fresh}} \leftarrow \text{MULTCONST}(\Delta, w)$ 
38: Re-randomize (circuit privacy)
39: sample  $v \leftarrow \text{ZO}(0.5)$ ,  $e_0, e_1 \leftarrow \text{DG}(\tau^2)$ ;  $c_{\text{fresh}} \leftarrow c_{\text{fresh}} + v \cdot \text{pk} + (e_0, e_1)$ 

40: return  $c_{\text{fresh}}$ 

```

of our non-leveled variant. We begin with fundamental homomorphic operations, such as addition and multiplication by constants, monomials, and polynomials, which serve as building blocks for approximating analytic functions. For simplicity in analysis, we assume that the error introduced by multiplication and refresh operations is bounded by a constant B^* . That is, if two ciphertexts (c_1, B_1) and (c_2, B_2) are multiplied and refreshed, the resulting ciphertext (c_0, B_0) satisfies the relation: $B_0 \leq \min\{B^*, B_{\text{mult}}\}$. If B_0 reaches threshold B^* , we set B_0 as B_{enc} with refresh operation. Analyzing basic constant operations, addition and multiplication by a

Algorithm 2 Power polynomial $f(x) = x^d$ for $d = 2^r$

```

1: Input: ciphertext  $c$ , bound  $B_c$ , exponent  $d = 2^r$ 
2: Output: ciphertext  $c_r$ , bound  $B_r$ 
3:  $c_0 \leftarrow c$ 
4:  $B_0 \leftarrow B_c$ 
5: for  $j = 1$  to  $r$  do
6:    $c_j \leftarrow \text{MULT}(c_{j-1}, c_{j-1})$ 
7:   if  $\frac{2\nu B_{j-1} + B_{j-1}^2 + 16\sqrt{3}\sigma^2 N}{\Delta} < B^*$  then
8:      $B_j \leftarrow \frac{2\nu B_{j-1} + B_{j-1}^2 + 16\sqrt{3}\sigma^2 N}{\Delta}$ 
9:   else
10:     $B_j \leftarrow B_{\text{enc}}$ ;  $c_j \leftarrow \text{REFRESH}(c_j)$ 
11:   end if
12: end for
13: return ( $c_r, B_r$ )

```

scalar constant $a \in R$, is shown as lemmas in the appendix, with extension to general polynomial evaluation further.

We describe Algorithm 2 for homomorphic evaluation of power functions $f(x) = x^d$, where d is a power-of-two integer, and analyze the error growth. At each squaring step, the ciphertext error approximately doubles, accumulates quadratic terms, and may trigger a refresh that resets noise to B_{enc} . For ciphertexts remaining below the threshold, the update rule is:

$$B_j = \frac{2\nu B_{j-1} + B_{j-1}^2 + 16\sqrt{3}\sigma^2 N}{\Delta},$$

where B_j is the noise bound after squaring the original ciphertext with noise bound B_{j-1} . We can extend Algorithm 3 to polynomial evaluations.

Let $f(x) = \sum_{j=0}^{\infty} a_j x^j$ be a real analytic function on an interval containing the plaintext range. By definition, such an f admits a convergent power series expansion around 0. We truncate this series to degree D so that the tail error $|\sum_{j>D} a_j m^j| \leq \epsilon$, for all plaintext values m in the domain of interest. For example, if $|m| \leq Q$, one can use a remainder bound from Taylor's Theorem to choose D such that

$$|a_{D+1}| \cdot |m|^{D+1} / (D+1)! < \epsilon \quad \text{or} \quad \sum_{j>D} |a_j| Q^j \leq \epsilon.$$

We evaluate the truncated polynomial $P_D(x) = \sum_{j=0}^D a_j x^j$ on ciphertexts using an adaptive-refresh strategy. For each monomial $a_j x^j$, the term x^j is computed via repeated squaring as in Algorithm 2. In practice, intermediate powers of x can be reused across terms.

When Δ is not set sufficiently large, each nontrivial multiplication rapidly approaches the threshold B^* , triggering a refresh operation that resets the noise to the fresh encryption bound B_{enc} , as shown in the proof of Lemma 6. We use this as the worst-case bound, though in practice, parameters can be chosen such that refresh is invoked less frequently, which requires a careful parametrization. The following theorem describes the evaluation of general analytic functions.

THEOREM 3. *Let function $f(x) = \sum_{j=0}^{\infty} a_j x^j$ be analytic on an interval containing the plaintext domains, and suppose (c_0, B_0) is a ciphertext encrypting m with initial noise $B_0 = B_{\text{enc}}$. Fix a refresh*

threshold B^* . Choose a truncation degree D depending on ϵ such that $|\sum_{j>D} a_j m^j| < \epsilon$ for all m in the domain. Then the adaptive Algorithm 2 outputs a ciphertext (c_f, B_f) encrypting the approximate value $P_D(m)$, such that the ciphertext noise satisfies

$$B_f \approx \min\left\{\sum_{j=0}^D |a_j| \cdot B_{enc} + \underbrace{\epsilon}_{\text{tail error}}, B^*\right\}.$$

Proof in Appendix C.8. $\square$

3.2.3 IND-CPA Security. We show that the proposed scheme achieves IND-CPA security under the RLWE assumption.

Threat Model: We consider a passive adversary in the chosen-plaintext attack (CPA) setting. In our threat model, the adversary $\mathcal{A}$ knows the public key pk of the encryption scheme and can request encryptions of plaintexts of its choice. The adversary's goal is to distinguish between the encryption of two messages of equal length under the same public key. This corresponds to the standard IND-CPA game, defined as follows. The adversary $\mathcal{A}$ may perform any number of encryption queries, polynomial in the security parameter, on plaintexts of its choice. Eventually, $\mathcal{A}$ outputs two messages m_0 and m_1 of equal length. A challenger then flips a random bit $b \in \{0, 1\}$, and returns an encryption of m_b to $\mathcal{A}$. The adversary outputs a guess bit b' for b . We say the encryption scheme is IND-CPA secure if for every probabilistic polynomial-time (PPT) adversary, the success probability is essentially $1/2$. Formally, for any efficient $\mathcal{A}$, its advantage is negligible in the security parameter λ : $|\Pr[b' = b] - \frac{1}{2}| \leq \text{negl}(\lambda)$.

Analysis: We assume the decisional RLWE problem is hard, even in the case where the secret s is drawn from a restricted distribution. The public key in the encryption scheme is of the form $pk = (b, a)$ where $a \in R_q$ is uniform and $b = -a \cdot s + e$. Thus, by design, the public key masks the secret key s as an RLWE instance. We next formalize and prove the IND-CPA security of our scheme under this assumption.

THEOREM 4. *Assuming the decisional RLWE problem over R_q is hard, the concrete construction of the given non-level scheme is IND-CPA secure.*

Proof in Appendix C.9. $\square$

3.2.4 Complexity and Memory Usage Analysis. We analyze the asymptotic cost of the proposed non-leveled CKKS. Implementation uses an RNS modulus $q = \prod_{i=1}^S q_i$. Relinearization/key-switching employs gadget base $T \geq 2$ with digit count $L = \lceil \log_T q \rceil$. When bootstrapping is enabled, the rounding polynomial has degree d and ρ slot rotations. With NTT, one degree- N multiplication in R_q costs $O(SN \log N)$. Per-algorithm costs appear in Table 1; for fixed S and L , runtime is quasi-linear in N and dominated by $\tilde{O}((1+L)SN \log N)$ from multiplications. Memory is linear in the number of stored ring polynomials: a degree-1 ciphertext holds two polys ($\approx 2N\Lambda$ bits); the evaluation key $\approx 2L N\Lambda$ bits; the boot key $\approx 2N\Lambda$ bits; rotation keys $\approx (2L) |\mathcal{G}| N\Lambda$ bits, where $|\mathcal{G}|$ is the number of Galois elements.

Table 1: Complexity Analysis of Non-Leveled CKKS Components (single fixed q)

Procedure	Time (ring ops)	Randomness (fresh draws)
KeyGen	$\tilde{O}((1+L)SN \log N)$	HWT(h) for s ; $U \times 2(a, a_0)$; $DG \times 2(e, e_0)$; bk: Bern + $DG \times 2$.
Ecd	$\tilde{O}(N \log N)$	none
Ded	same as Ecd	none
Enc	$\tilde{O}(SN \log N)$	Bern(v); $DG \times 2(e_0, e_1)$
Dec	$O(SN)$	none
Add	$O(SN)$	none
Mult (incl. rlin)	$\tilde{O}((1+L)SN \log N)$	none (uses evk)
Refresh (bootstrap)	$\tilde{O}((\sqrt{d}+\rho)SN \log N)$	optional flooding: Bern + $DG \times 2$

Params: S = #RNS limbs ($q = \prod q_i$), $L = \lceil \log_T q \rceil$ (gadget digits), d = degree of rounding polynomial, ρ = #slot rotations. Each ring polynomial stores $\approx N\Lambda$ bits with $\Lambda = \log_2 q$. The boot key (bk) is a single ciphertext $Enc(s)$.

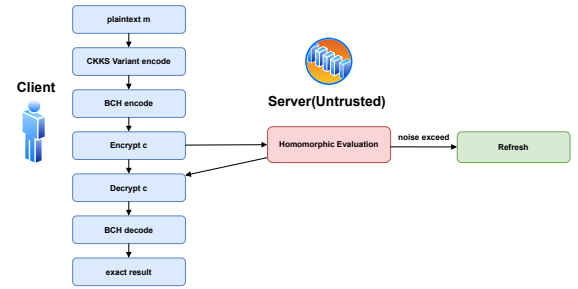


Figure 1: Ring-BCH pre-/post-processing entirely in $R = \mathbb{Z}_q[X]/(X^N+1)$.

3.3 Fault-Tolerance via Ring-BCH

While CKKS is attractive for privacy-preserving ML due to its support for approximate arithmetic and SIMD packing, practical deployments demand robust decryption in the presence of noise from both homomorphic evaluation and real-world transport. A single RNS-limb fault can compromise confidentiality, and in production ML pipelines, such as streaming/RPC, federated aggregation, batched inference, transient bit flips during data transit can derail decoding and force costly retries or re-transmissions. To address these issues, we introduce an algebraic error-correcting layer that stays inside the plaintext ring and composes cleanly with non-leveled CKKS.

Concretely, inspired by [2, 21, 36], we instantiate a Ring-BCH code as an ideal of $R = \mathbb{Z}_{2^k}[X]/(X^N+1)$, ensuring closure under the ring operations that underlie linear homomorphic evaluation. We further employ a code-preserving automorphism interleaver to disperse low-significance error clusters without leaving the code. This reliability layer supports privacy-preserving linear algebra and integrates into end-to-end ML workflows; it can be extended to broader HE circuits by exploiting the algebraic structure of R . Figure 1 illustrates the workflow: messages are segmented into R -polynomial blocks, encoded by a Ring-BCH ideal, interleaved via a ring automorphism, encrypted, and finally decoded after decryption.

3.3.1 Construction Overview. We work in the 2-adic setting with odd N such that $N \mid (2^m-1)$, so X^N+1 splits over $\mathbb{F}_{2^m}$. Using the standard binary BCH construction with designed distance δ and consecutive locators, we obtain a generator $g_2 \in \mathbb{F}_2[X]$ dividing

$X^N + 1$. By Hensel lifting there is a unique monic $g_k \in \mathbb{Z}_{2^k}[X]$ with $g_k \bmod 2 = g_2$ and $g_k \mid (X^N + 1)$. We define the Ring-BCH code as the principal ideal (g_k) in $R = \mathbb{Z}_{2^k}[X]/(X^N + 1)$ and use systematic encoding with payload dimension $K = N - \deg g_k$.

This choice is operationally convenient for PPML pipelines. Because (g_k) is an ideal of R , it is closed under ring addition and multiplication, so linear homomorphic evaluation preserves code membership and does not interfere with encrypted linear algebra. Reducing modulo 2 recovers the underlying binary BCH code with designed distance at least δ , and standard Gray-map arguments transfer corresponding Lee/Hamming distance lower bounds over $\mathbb{Z}_{2^k}$, providing deterministic correction capability. Finally, ring automorphisms act as coordinate permutations that preserve the ideal, dispersing burst errors before decoding without leaving the code [18]. In practice, we pack K slots per block, evaluate homomorphically as usual, and decode at the edge with low overhead while retaining the CKKS programming model.

3.3.2 Systematic encoding and decoding. For a message $u(X)$ with $\deg u < K$, define the systematic codeword

$$c(X) = u(X)X^{N-K} + (u(X)X^{N-K} \bmod g_k(X)) \in C.$$

Decoding proceeds by a mod-2 pass (syndromes, BM/EEA, Chien, Forney) followed by Hensel lifting of the error values to 2^k and inversion of the systematic map. All steps run in R . A concrete encoder/decoder appears in Algorithm 3 (appendix A).

3.3.3 Interleaving. CKKS error tends to concentrate in low significance within *coefficients*. To de-cluster these errors across code positions, we use a ring automorphism as an algebraic interleaver. Figure 2 illustrates the essence of this automorphism. For any odd s with $\gcd(s, N) = 1$, the map $\sigma_s : R \rightarrow R, X \mapsto X^s$, is a ring automorphism since $(X^s)^N = (X^N)^s = (-1)^s = -1$ in R . Because $C = (g_k)$ is an ideal, $\sigma_s(C) = C$. We use σ_s (and optionally cyclic shifts X^t) as *code-preserving* interleavers that permute coefficient indices $i \mapsto si \bmod N$ while keeping closure under ring addition/multiplication. At the sender, apply σ_s to each codeword before encryption; at the receiver, apply $\sigma_{s^{-1}}$ prior to decoding.

3.3.4 Payload Layout in R . Let M be the payload length in bits. Fix $\text{BCH}(n, k_{\text{BCH}}, t) = (127, 101, 3)$ and select $n = 127$ (or embed a degree-127 factor in a larger odd N and restrict to that component). Segment the payload into $h := \lceil \frac{M}{k_{\text{BCH}}} \rceil$ blocks $u^{(i)} \in \{0, 1\}^{k_{\text{BCH}}}$, map each block to a polynomial $u^{(i)}(X)$ over $\mathbb{Z}_{2^k}$ by placing the bits in the LSBs of the K message positions, encode $c^{(i)} = \text{RINGBCH_ENCODE}(u^{(i)}) \in C \subset R$, and optionally interleave via $c^{(i)} \leftarrow \sigma_s(c^{(i)})$. Each $c^{(i)}$ is then encrypted as a plaintext in R . On the way back, decrypt, apply $\sigma_{s^{-1}}$, decode with RINGBCH_DECODE , and concatenate the recovered $u^{(i)}$'s. Algorithm 4 in the appendix shows this design concretely.

3.3.5 IND-CPA Security. Let $\mathcal{S}$ be the base scheme over R and $\mathcal{S}^*$ the scheme that composes Ring-BCH pre-encoding before encryption and Ring-BCH post-decoding after decryption. The Ring-BCH layer is public, deterministic, and invertible on codewords; therefore, it is a benign pre-/post-processing.

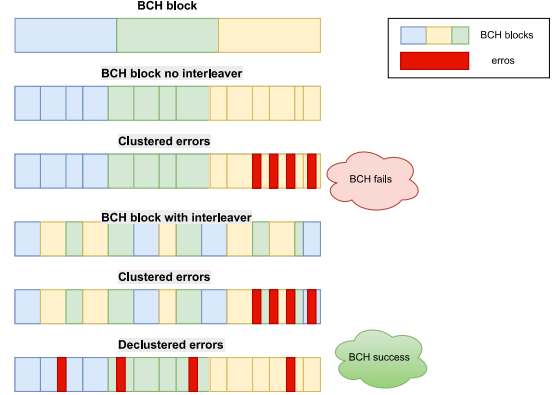


Figure 2: Interleaving de-clusters errors: yellow/blue/green are BCH blocks; red stripes are localized errors.

THEOREM 5. For every PPT adversary $\mathcal{A}$ against $\mathcal{S}^*$ there exists a PPT adversary $\mathcal{B}$ against $\mathcal{S}$ such that

$$\text{Adv}_{\mathcal{S}^*}^{\text{ind-cpa}}(\mathcal{A}) \leq 2 \text{Adv}_{\mathcal{S}}^{\text{ind-cpa}}(\mathcal{B}).$$

Proof sketch in Appendix C.10.

4 Evaluation

We evaluate our framework from a system perspective, focusing on its suitability for web-scale and federated machine learning deployments. Specifically, we ask:

- **Q1: Latency and throughput.** Does the non-leveled CKKS variant reduce the overhead of homomorphic operations compared to standard CKKS in end-to-end ML workloads?
- **Q2: Robustness.** How effectively does the Ring-BCH layer correct in-transit faults (bit flips, packet loss) that occur in distributed web environments?
- **Q3: Accuracy preservation.** Does our approach maintain model accuracy across common privacy-preserving ML tasks under realistic noise and error conditions?

4.1 Experimental Setup

We implemented our system on top of HELIB [24] with custom modules for non-leveled CKKS and Ring-BCH encoding/decoding. All benchmarks were run on a Linux host with an AMD Ryzen 7 (8 cores/16 threads) CPU and 32 GB RAM, running Ubuntu 22.04. To emulate federated learning, we deployed client instances that perform local encryption and transmission, while the server aggregates and evaluates ciphertexts. We locally implemented the proposed framework by extending the HELIB library with custom modules while making minimal changes to the library's core. Specifically, we introduced two new source files `simple_nonleveled_ckks.h` and `simple_nonleveled_ckks.cpp`, which define a class to support non-leveled CKKS operations. These new classes encapsulate the entire encryption-scheme workflow, including key generation, encoding, encryption, homomorphic evaluation, and decryption, all tailored for single-level data. We modified a few existing HELIB

Table 2: Datasets for Application

Workload	Datasets
Encrypted inference	MNIST (60k/10k), CIFAR-10 (50k/10k)
Federated aggregation	MNIST/CIFAR-10 with Dirichlet non-IID ($\alpha \in \{0.1, 0.3\}$)
Streaming analytics	UCI Household Power (minute-level);

Table 3: Models/operations and primary metrics (tested at $N \in \{1024, 2048, 4096, 8192\}$).

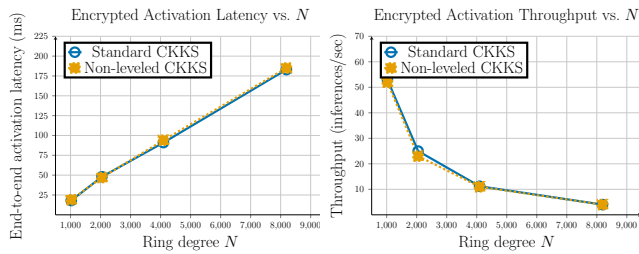
Workload	Model / Operation	Primary Metrics
Encrypted inference	LogReg; 2-layer MLP	Top-1; end-to-end latency; p99; throughput
Federated aggregation	Encrypted mean of client updates per round	Agg. latency vs. clients; p99 tail; bytes; rel. ℓ_2 error
Streaming analytics	Sliding-window mean	memory vs. W ; failure vs. p ; ℓ_∞ deviation

utility headers (e.g., `binio.h` and `io.h`) to accommodate our custom data types, while most of the added functionality resides in the new files within the benchmarks/ directory, keeping the core codebase intact.

Overall, we added roughly 1,100 lines of code across two new files, `simple_nonleveled_ckks.h` and `simple_nonleveled_ckks.cpp`, and touched fewer than 40 lines in three existing HELIB utility headers (`binio.h`, `io.h`, and `NumbTh.h`). Because virtually all new logic resides in the two new files under benchmarks/, the core HELIB codebase remains intact, making the modification easy to audit, maintain, and port. We evaluate three representative workloads, as shown in Table 3, with the data sets in Table 2.

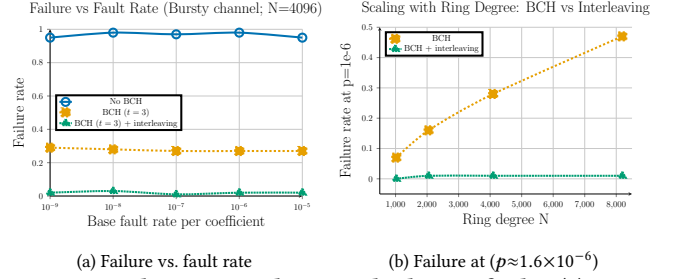
4.2 End-to-End Latency and Throughput

We measure wall-clock latency of full encrypted inference pipelines. Each experiment consists of encrypting inputs, performing homomorphic evaluation of a cubic activation $a_3x^3 + a_2x^2 + a_1x + a_0$ (two ciphertext-ciphertext multiplications and three additions), a single scale-management step (Rescale or Refresh), and decryption.

**(a) Latency vs. N** **(b) Throughput vs. N** **Figure 3: End-to-end encrypted activation ($2 \times \text{Mult} + 1 \times \text{scale} + 3 \times \text{Add}$).**

Using the measured primitive costs, the end-to-end latency (ms) for Standard vs. Non-Leveled at $N \in \{1024, 2048, 4096, 8192\}$ is:

1024 : 18.99 vs 19.56 (NL +0.57 ms, -3.0%),
 2048 : 42.33 vs 43.72 (NL +1.39 ms, -3.3%),
 4096 : 91.54 vs 91.23 (NL -0.31 ms, +0.34%),
 8192 : 197.63 vs 195.81 (NL -1.82 ms, +0.92%).

**(a) Failure vs. fault rate****(b) Failure at $(p \approx 1.6 \times 10^{-6})$** **Figure 4: Robustness and cost under bursty faults. (a) Interleaving keeps failure < 0.5%. (b) Failure vs. N without/with interleaving.**

The corresponding throughput (inferences/sec) is:

1024 : 52.66 vs 51.12 (-3.0%), 2048 : 23.62 vs 22.86 (-3.3%),
 4096 : 10.92 vs 10.96 (+0.4%), 8192 : 5.06 vs 5.11 (+1.0%).

Figure 3 visualizes these trends. Although REFRESH is costlier than RESCALE in isolation, its relative overhead shrinks with N and is amortized by the multiply-dominated pipeline, yielding parity at $N=4096$ and a small advantage at $N=8192$.

Amortization. At $N=4096$, REFRESH costs $\approx 1,105 \mu\text{s}$. If invoked every d multiplications, the amortized penalty is $\leq 1105/d \mu\text{s}$ per multiply; for $d \geq 4$ this is $\leq 0.28 \text{ ms}$, i.e., $\leq 0.6\%$ of a $\sim 45 \text{ ms}$ multiply. This explains the negligible end-to-end gap at $N=4096$ and the slight throughput gains at $N=8192$.

4.3 Robustness to Network Faults

We evaluate Q2 under correlated transmission errors using a Gilbert-Elliott channel. In the Good state, each coefficient flips with base rate $p \in [10^{-9}, 10^{-5}]$; in the Bad state (packet/word corruption) the flip rate is $p_{\text{bad}} = 2 \times 10^{-2}$ with average burst length $L_{\text{bad}} = 64$ coefficients and average Good-run length $L_{\text{good}} = 800$. Ciphertexts carry BCH(127, 101, 3) codewords; at ring degree N we use $h = \lceil N/101 \rceil$ blocks (e.g., $h=41$ at $N=4096$). A trial succeeds iff every block has $\leq t=3$ symbol errors. We compare No BCH, BCH (no interleaver), and BCH+Interleave (automorphism-based stride interleaver). Each point averages 600 Monte Carlo trials.

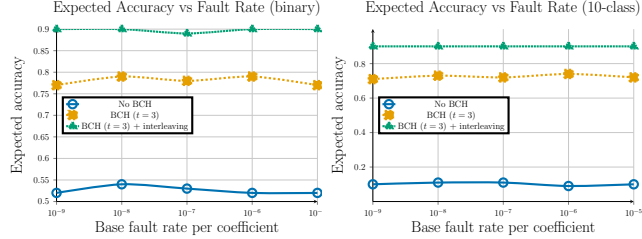
$N=4096$ across p . Figure 4(a) shows that No BCH fails in 95.7–98.2% of trials across $p \in [10^{-9}, 10^{-5}]$. BCH without interleaver improves to 25.3–30.2% failure, while BCH+Interleave is stable at 0.17–0.50%.

Scaling with N . At a representative base rate $p \approx 1.6 \times 10^{-6}$, BCH without interleaver failure grows from 6.7% at $N=1024$ to 16.3% at 2048, 29.5% at 4096, and 47.8% at 8192; in contrast, BCH+Interleave stays near 0.17%–0.33% across all N (Fig. 4(b)). With 600 trials, 95% Clopper-Pearson intervals are shown in Table 4. These results quantify the intuition that with more BCH blocks, the probability that some block exceeds the t -budget rises quickly unless bursts are dispersed.

Overhead vs. N . At $N=4096$ the total is $4,160 \mu\text{s}$, decomposed as encode $1,690 \mu\text{s}$, permute $585 \mu\text{s}$, inverse $552 \mu\text{s}$, and decode $1,332 \mu\text{s}$. This adds only a few milliseconds to multiply-dominated inference pipelines (§4) while yielding two orders of magnitude lower failure under bursts. At $N=8192$, the total overhead is $\approx 8.3 \text{ ms}$, which is very economical.

Table 4: Failure at $p \approx 1.6 \times 10^{-6}$ (95% CIs in parentheses; 600 trials).

N	BCH (%)	BCH+Interleave (%)
1024	6.7 (4.8–9.0)	0.17 (0.00–0.93)
2048	16.3 (13.5–19.5)	0.33 (0.04–1.20)
4096	29.5 (25.9–33.3)	0.17 (0.00–0.93)
8192	47.8 (43.8–51.9)	0.17 (0.00–0.93)

(a) Binary, $N=4096$ (b) 10-class, $N=4096$ **Figure 5: Expected accuracy vs. base fault rate p using failure rates from Fig. 4(a) and the model $A_{\text{exp}} = (1-f)A_{\text{plain}} + fA_{\text{rand}}$. We set $A_{\text{plain}}=0.90$ (binary) and 0.98 (10- class) for illustration.**

Takeaways. (i) Without protection, virtually any burst derails decryption. (ii) BCH alone helps but degrades with ring dimension because the chance that one block exceeds t grows with $h=\lceil N/101 \rceil$. (iii) Automorphism-based interleaving keeps per-block errors within t , making failure almost *flat* in N , at sub-percent levels under bursty faults, with millisecond-level overhead.

4.4 Accuracy Preservation in ML Tasks

We quantify the effect of REFRESH and Ring-BCH on inference accuracy. Following prior CKKS work, we evaluate two representative workloads: (i) a binary classification task and (ii) a 10-class task, e.g., a small MLP. Let A_{plain} denote plaintext test accuracy and A_{rand} the accuracy of a random guesser (0.5 for binary; 0.1 for 10- class). Under encryption, approximation error from REFRESH is negligible; the dominant source of degradation under network faults is *decode failure*. If a trial fails, we conservatively assume a random prediction. Thus, the expected accuracy is

$$A_{\text{exp}} = (1-f)A_{\text{plain}} + fA_{\text{rand}},$$

where f is the measured failure rate from §4.3.

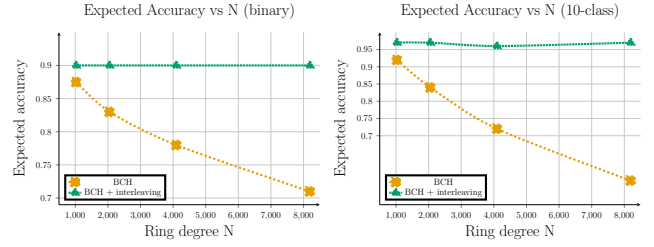
Using the bursty channel from §4.3, Fig. 5 shows that BCH with Interleave tracks the plaintext baseline: the expected accuracy remains ≥ 0.895 (binary) and ≥ 0.975 (10- class) across $p \in [10^{-9}, 10^{-5}]$. In contrast, without interleaving, clustered errors reduce expected accuracy to 0.78–0.80 (binary) and 0.72–0.76 (10- class), reflecting the measured 25–30% failure in Fig. 4(a). The *No BCH* baseline collapses to the random guesser whenever bursts occur, yielding ~ 0.51 (binary) and ~ 0.12 (10- class).

Scaling with N . At $p \approx 1.6 \times 10^{-6}$, expected accuracy for BCH without interleaver declines as N grows: binary drops from

$$0.90 \rightarrow 0.83 \rightarrow 0.78 \rightarrow 0.71.$$

at $N=1024, 2048, 4096, 8192$; 10- class drops from

$$0.92 \rightarrow 0.84 \rightarrow 0.72 \rightarrow 0.56.$$

(a) Binary, $p \approx 1.6 \times 10^{-6}$ (b) 10- class, $p \approx 1.6 \times 10^{-6}$ **Figure 6: Expected accuracy vs. ring degree N at a representative base rate ($p \approx 1.6 \times 10^{-6}$). BCH (no interleaver) degrades with N because the number of blocks $h=\lceil N/101 \rceil$ grows, increasing the chance that some block exceeds the $t=3$ budget; interleaving keeps accuracy near the plaintext baseline.**

In contrast, BCH+Interleave remains flat within <0.5 points of plaintext across all N .

Takeaways. (i) REFRESH contributes negligible approximation error in our pipelines; accuracy is governed by decode success. (ii) BCH without interleaving becomes unreliable as N increases (more blocks $\Rightarrow$ higher chance some block exceeds t). (iii) Automorphism-based interleaving keeps *expected* accuracy within $<0.5\%$ of plaintext across fault rates and ring sizes, consistent with Fig. 5 and Fig. 6.

5 Conclusion and Future Work

This paper presented an FHE framework for web services that adds two key features to the CKKS scheme. The first is a non-leveled, fixed-scale design that replaces the standard Rescale operation with a lightweight Refresh operation. This approach simplifies the programming model by removing the need for depth planning and results in slower noise growth during multiplication. The second feature is an error-correction layer using a Ring-BCH outer code and automorphism-based interleaving. This design enables the exact recovery of ciphertexts corrupted during network transport, is compatible with homomorphic operations, and does not weaken the underlying IND-CPA security of the scheme. Our framework's practicality was confirmed through evaluations on three web workloads: encrypted inference on MNIST/CIFAR-10, federated aggregation, and streaming analytics. Across these tasks, the system achieves latency comparable to leveled baselines while reducing the failure rate under bursty network faults to below 0.5%, all with an accuracy loss of less than 0.5 percentage points compared to plaintext.

Future Works. We plan several directions for optimization and investigation. This includes further optimizing the underlying ring arithmetic using structure-aware techniques [10], refining the re-linearization process for the Refresh operation, and preprocessing [9, 47, 48]. We will also seek to accelerate the Ring-BCH encoding and decoding with vectorized algorithms and explore potential hardware offloading [28]. Finally, we will benchmark our framework against recent CKKS optimizations and investigate the integration of lightweight, TFHE-style bootstrapping under unified security analyses [3, 4, 12, 34].

References

- [1] Sajjad Akherati and Xinmiao Zhang. 2023. Low-complexity ciphertext multiplication for CKKS homomorphic encryption. *IEEE Transactions on Circuits and Systems II: Express Briefs* 71, 3 (2023), 1396–1400.
- [2] Salah A Aly, Andreas Klappenecker, and Pradeep Kiran Sarvepalli. 2007. On quantum and classical BCH codes. *IEEE Transactions on Information Theory* 53, 3 (2007), 1183–1188.
- [3] Youngjin Bae, Jung Hee Cheon, Guillaume Hanrot, Jai Hyun Park, and Damien Stehlé. 2024. Plaintext-ciphertext matrix multiplication and FHE bootstrapping: fast and fused. In *Annual International Cryptology Conference*. Springer, 387–421.
- [4] Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, and Damien Stehlé. 2024. Bootstrapping bits with CKKS. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 94–123.
- [5] Youngjin Bae, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. 2024. Bootstrapping small integers with CKKS. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 330–360.
- [6] Raj Chandra Bose and Dwijendra K Ray-Chaudhuri. 1960. On a class of error correcting binary group codes. *Information and control* 3, 1 (1960), 68–79.
- [7] Jean-Philippe Bossuat, Christian Mouchet, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2021. Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 587–617.
- [8] Zvika Brakerski. 2012. Fully homomorphic encryption without modulus switching from classical GapsVP. In *Annual cryptography conference*. Springer, 868–886.
- [9] Andreas Brüggemann, Robin Hundt, Thomas Schneider, Ajith Suresh, and Hossein Yalame. 2023. FLUTE: fast and secure lookup table evaluations. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 515–533.
- [10] Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Damien Stehlé. 2023. Homomorphic multiple precision multiplication for CKKS and reduced modulus consumption. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 696–710.
- [11] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. Bootstrapping for approximate homomorphic encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 360–384.
- [12] Jung Hee Cheon, Guillaume Hanrot, Jongmin Kim, and Damien Stehlé. 2025. Ship: A shallow and highly parallelizable ckks bootstrapping algorithm. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 398–428.
- [13] Jung Hee Cheon, Minsik Kang, and Jai Hyun Park. 2025. Towards Lightweight CKKS: On Client Cost Efficiency. *Cryptology ePrint Archive* (2025).
- [14] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic encryption for arithmetic of approximate numbers. In *International conference on the theory and application of cryptology and information security*. Springer, 409–437.
- [15] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachene. 2016. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In *international conference on the theory and application of cryptology and information security*. Springer, 3–33.
- [16] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. 2020. TFHE: fast fully homomorphic encryption over the torus. *Journal of Cryptology* 33, 1 (2020), 34–91.
- [17] Pierre-Emmanuel Clet, Oana Stan, and Martin Zuber. 2021. BFV, CKKS, TFHE: Which one is the best for a secure neural network evaluation in the cloud?. In *International Conference on Applied Cryptography and Network Security*. Springer, 279–300.
- [18] Yong Deng. 2022. Random permutation set. *International Journal of Computers Communications & Control* 17, 1 (2022).
- [19] Junfeng Fan and Frederik Vercauteren. 2012. Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive* (2012).
- [20] Shereen Mohamed Fawaz, Nahla Belal, Adel Elrefaey, and Mohamed Waleed Fakhr. 2021. A comparative study of homomorphic encryption schemes using microsoft seal. In *Journal of Physics: Conference Series*, Vol. 2128. IOP Publishing, 012021.
- [21] George Forney. 2003. On decoding BCH codes. *IEEE Transactions on information theory* 11, 4 (2003), 549–557.
- [22] Craig Gentry. 2009. *A fully homomorphic encryption scheme*. Stanford university.
- [23] Yanwei Gong, Xiaolin Chang, Jelena Mišić, Vojislav B Mišić, Jianhua Wang, and Haoran Zhu. 2024. Practical solutions in fully homomorphic encryption: a survey analyzing existing acceleration methods. *Cybersecurity* 7, 1 (2024), 5.
- [24] Shai Halevi and Victor Shoup. 2014. Algorithms in helib. In *Annual Cryptology Conference*. Springer, 554–571.
- [25] Boyoung Han, Hojune Shin, Yeonghyeon Kim, Jina Choi, and Younho Lee. 2024. HEaaN-NB: Non-Interactive Privacy-Preserving Naive Bayes Using CKKS for Secure Outsourced Cloud Computing. *IEEE Access* (2024).
- [26] Intak Hwang, Yisol Hwang, Miran Kim, Dongwon Lee, and Yongsoo Song. 2025. Provably Secure Approximate Computation Protocols from CKKS. *Cryptology ePrint Archive* (2025).
- [27] Mario Larch, Joschka Wanner, Yoto V Yotov, and Thomas Zylkin. 2019. Currency unions and trade: A PPML re-assessment with high-dimensional fixed effects. *Oxford Bulletin of Economics and Statistics* 81, 3 (2019), 487–510.
- [28] Jaehyeok Lee, Phap Ngoc Duong, and Hanho Lee. 2023. Configurable encryption and decryption architectures for CKKS-based homomorphic encryption. *Sensors* 23, 17 (2023), 7389.
- [29] Min Liang and Li Yang. 2015. Quantum fully homomorphic encryption scheme based on quantum fault-tolerant construction. *arXiv preprint arXiv:1503.04061* (2015).
- [30] Rasoul Akhavan Mahdavi, Abdulrahman Daa, and Florian Kerschbaum. 2023. HE is all you need: Compressing FHE Ciphertexts using Additive HE. *arXiv preprint arXiv:2303.09043* (2023).
- [31] Lakshmi Likhitha Mankali, Mohammed Nabeel, Faiq Raees, Michail Maniatakos, Ozgur Sinanoglu, and Johann Knechtel. 2025. {GlitchFHE}: Attacking Fully Homomorphic Encryption Using Fault Injection. In *34th USENIX Security Symposium (USENIX Security 25)*. 8481–8500.
- [32] Erik Mårtensson. 2019. The asymptotic complexity of coded-BKW with sieving using increasing reduction factors. In *2019 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2579–2583.
- [33] Federico Mazzone, Maarten Everts, Florian Hahn, and Andreas Peter. 2025. Efficient Ranking, Order Statistics, and Sorting under {CKKS}. In *34th USENIX Security Symposium (USENIX Security 25)*. 8541–8558.
- [34] Daniele Micciancio and Yuriy Polyakov. 2021. Bootstrapping in FHEW-like cryptosystems. In *Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography*. 17–28.
- [35] Seonhong Min, Joon-Woo Lee, and Yongsoo Song. 2025. Enhanced CKKS Bootstrapping with Generalized Polynomial Composites Approximation. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*. 1–12.
- [36] Jorge Castiñeira Moreira and Patrick Guy Farrell. 2006. *Essentials of error-control coding*. John Wiley & Sons.
- [37] Christian Vincent Mouchet, Jean-Philippe Bossuat, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2020. Lattigo: A multiparty homomorphic encryption library in go. In *Proceedings of the 8th Workshop on Encrypted Computing and Applied Homomorphic Cryptography*. 64–70.
- [38] Kurt Rohloff and David Bruce Cousins. 2014. A scalable implementation of fully homomorphic encryption built on NTRU. In *International Conference on Financial Cryptography and Data Security*. Springer, 221–234.
- [39] Goro Shimura. 1963. Arithmetic of alternating forms and quaternion hermitian forms. *Journal of the Mathematical Society of Japan* 15, 1 (1963), 33–65.
- [40] IlKwon Sohn, Boseon Kim, Kwangil Bae, and Wonhyuk Lee. 2024. Error correctable efficient quantum homomorphic encryption using Calderbank-Shor-Steane codes. *arXiv preprint arXiv:2401.08059* (2024).
- [41] Akshitha Sriraman, Sihang Liu, Sinan Gunbay, Shan Su, and Thomas F Wenisch. 2017. Deconstructing the tail at scale effect across network protocols. *arXiv preprint arXiv:1701.03100* (2017).
- [42] Jonathan Takeshita, Nirajan Koirala, Colin McKechney, and Taeho Jung. 2025. HE-Profiler: an in-depth profiler of approximate homomorphic encryption libraries: J. Takeshita et al. *Journal of Cryptographic Engineering* 15, 2 (2025), 14.
- [43] Zejiu Tan, Junping Wan, Zoe L Jiang, Jingjing Fan, Man Ho Au, and Siu Ming Yiu. 2025. High-precision homomorphic modular reduction for CKKS bootstrapping. In *Australasian Conference on Information Security and Privacy*. Springer, 232–251.
- [44] Olamide Timothy Tawose, Jun Dai, Lei Yang, and Dongfang Zhao. 2023. Toward efficient homomorphic encryption for outsourced databases through parallel caching. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–23.
- [45] Marten Van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. 2010. Fully homomorphic encryption over the integers. In *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 24–43.
- [46] Zhiwei Wang, Peinan Li, Rui Hou, Zhihao Li, Jiangfeng Cao, XiaoFeng Wang, and Dan Meng. 2023. HE-Booster: an efficient polynomial arithmetic acceleration on GPUs for fully homomorphic encryption. *IEEE Transactions on Parallel and Distributed Systems* 34, 4 (2023), 1067–1081.
- [47] Arantxa Zapico, Vitalik Buterin, Dmitry Khovratovich, Mary Maller, Anca Nitulescu, and Mark Simkin. 2022. Caulk: Lookup arguments in sublinear time. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 3121–3134.
- [48] Arantxa Zapico, Ariel Gabizon, Dmitry Khovratovich, Mary Maller, and Carla Ràfols. 2022. Baloo: Nearly optimal lookup arguments. *Cryptology ePrint Archive* (2022).
- [49] Dongfang Zhao. 2023. Silca: Singular Caching of Homomorphic Encryption for Outsourced Databases in Cloud Computing. *arXiv preprint arXiv:2306.14436* (2023).
- [50] Dongfang Zhao. 2025. Compile-Time Fully Homomorphic Encryption: Eliminating Online Encryption via Algebraic Basis Synthesis. *Cryptography (2410-387X)* 9, 2 (2025).
- [51] Dongfang Zhao. 2025. A Note on Efficient Privacy-Preserving Similarity Search for Encrypted Vectors. *arXiv preprint arXiv:2502.14291* (2025).

Algorithm 3 RINGBCH_ENCODE and RINGBCH_DECODE

```

1: Setup:  $R = \mathbb{Z}_{2^k}[X]/(X^N+1)$  with  $N$  odd; monic  $g_k \mid (X^N+1)$ ,
    $g_k \bmod 2 = g_2$ ; code  $C = (g_k) \subset R$ ;  $K = N - \deg g_k$ .
2: Procedure RINGBCH_ENCODE( $u$ )
3: Input:  $u \in \mathbb{Z}_{2^k}[X]$ ,  $\deg u < K$ 
4:  $w \leftarrow uX^{N-K}$ ;  $r \leftarrow w \bmod g_k$  in  $\mathbb{Z}_{2^k}[X]$ 
5:
6: return  $c \leftarrow (w + r) \bmod (X^N+1) \in C$ 
7: Procedure RINGBCH_DECODE( $r$ )
8: Input:  $r \in R$  (received)
9:  $\tilde{r} \leftarrow r \bmod 2$ ;  $S_j \leftarrow \tilde{r}(\tilde{a}^{b+j})$  for  $j = 0, \dots, \delta - 2$ 
10:  $(\Lambda, \Omega) \leftarrow \text{KEYEQUATIONSOLVE}(S)$ ;  $I \leftarrow \text{CHIENSEARCH}(\Lambda)$ 
11:  $e^{(1)} \leftarrow \text{FORNEY}(\Lambda, \Omega)$  (support  $I$  and mod-2 error values)
12: for  $\ell = 1$  to  $k-1$  do
13:    $e^{(2^{\ell+1})} \leftarrow \text{LIFTDIGIT}\left(r, \sum_{j=0}^{\ell-1} 2^j e^{(2^{j+1})}, I\right)$ 
14: end for
15:  $e \leftarrow \sum_{\ell=0}^{k-1} 2^\ell e^{(2^{\ell+1})}$ ;  $c \leftarrow r - e$ 
16:  $u \leftarrow \text{SYSTEMATICINVERSE}(c)$  (divide by  $X^{N-K}$  and reduce mod  $g_k$ )
17:
18: return  $c$  (and  $u$ )

```

Algorithm 4 SEGMENTENCODEINR and DECODECONCATINR (no external packing)

```

1: Setup:  $n = 127$ , BCH(127, 101, 3) lifted to  $g_k$ ; ideal  $C = (g_k) \subset R$ ;
   automorphism  $\sigma_s$  with  $\gcd(s, N) = 1$ .
2: Procedure SEGMENTENCODEINR( $m_{\text{bin}} \in \{0, 1\}^M$ )
3: Output: ciphertexts  $\{C^{(i)}\}_{i=0}^{h-1}$ 
4:  $h \leftarrow \lceil M/101 \rceil$ ; pad  $m_{\text{bin}}$  to length 101h
5: for  $i = 0$  to  $h-1$  do
6:    $u^{(i)} \leftarrow m_{\text{bin}}[i \cdot 101..(i+1) \cdot 101-1]$ 
7:    $u^{(i)}(X) \leftarrow \text{PLACE LSB}(u^{(i)})$  on the  $K=101$  message positions
8:    $c^{(i)} \leftarrow \text{RINGBCH\_ENCODE}(u^{(i)}(X))$ 
9:    $c^{(i)} \leftarrow \sigma_s(c^{(i)})$  (code-preserving interleave)
10:   $C^{(i)} \leftarrow \text{ENC}(c^{(i)})$ 
11: end for
12:
13: return  $\{C^{(i)}\}_{i=0}^{h-1}$ 
14: Procedure DECODECONCATINR( $\{C^{(i)}\}_{i=0}^{h-1}$ )
15: Output:  $\hat{m}_{\text{bin}} \in \{0, 1\}^M$ 
16: for  $i = 0$  to  $h-1$  do
17:    $r^{(i)} \leftarrow \text{DEC}(C^{(i)})$ 
18:    $\hat{c}^{(i)} \leftarrow \sigma_{s^{-1}}(r^{(i)})$ 
19:    $\hat{u}^{(i)} \leftarrow \text{RINGBCH\_DECODE}(\hat{c}^{(i)})$ 
20: end for
21:
22: return  $\hat{u}^{(0)} \parallel \dots \parallel \hat{u}^{(h-1)}[0..M-1]$ 

```

[52] Wenpeng Zhao, Qidong Chen, Yijie Wang, Haichun Zhang, Zhaojun Lu, and Gang Qu. 2024. A Survey on FPGA-based Accelerators for CKKS. In *2024 IEEE International Test Conference in Asia (ITC-Asia)*. IEEE, 1–6.

A Concrete Algorithms

Algorithm 3 describes the encoding and decoding process for the Ring BCH layer, and Algorithm 4 describes segment encoding and decoding with a BCH code of parameter (127, 101, 3) and an algebraic structure preserving permutation.

B Correctness of the Non-Leveled Scheme

Sketch. Let $R_q = \mathbb{Z}[X]/(X^N+1, q)$ and encode at fixed scale Δ ; every ciphertext is an RLWE sample $(b, a) = (m + e - as, a)$ with

small noise e . Maintain the invariant $\mathcal{I}(c) : \text{scale}(c) = \Delta$ and $\|e(c)\|_\infty \leq B$ with $\Delta B \leq q/8$. Enc/Dec correctness follows since decryption yields $m + e'$ and, under the bound, rounding in the canonical embedding recovers slots up to standard CKKS approximation. Add preserves scale and sums noises; Mult+relinearization produces pre-refresh noise B_{mult} that is (standardly) a bilinear-plus-linear function of input bounds and gadget noise and is the only step that can threaten $\Delta B \leq q/8$. **Refresh** homomorphically forms $\text{Enc}(m+e)$, applies a low-degree polynomial $R(x) \approx \text{round}(x/\Delta)$, rescales back to Δ , and re-randomizes; choosing degree/precision and flooding variance ensures a post-refresh bound $B' \leq B_{\text{enc}}$ with $\Delta B' \leq q/8$, restoring $\mathcal{I}$ independent of prior depth. Because the plaintext Ring-BCH code $(g_k) \subset R$ is an ideal, R -linear evaluation preserves code membership; automorphism interleaving merely permutes coordinates, and BCH decoding corrects whenever each block has $\leq t$ errors after transport. Scheduling **Refresh** whenever $B_{\text{mult}} \geq B^*$ yields, by induction over the circuit, that $\mathcal{I}$ holds at every step and the final slot error is $\epsilon_{\text{enc}} + n_{RE} + \epsilon_{\text{lin}}$; all ciphertexts remain RLWE samples, so semantic security is unchanged.

C Statement and Proof of Lemmas, Propositions, and Theorems

Assume that a polynomial $a(X) \in R = \mathbb{Z}[X]/(\Phi_M(X))$ is sampled from a uniform distribution or a discrete Gaussian distribution, and its nonzero coefficients are independently and identically distributed. Since $a(\zeta_M)$ is the inner product of the coefficient vector of a and the fixed vector $(1, \zeta_M, \dots, \zeta_M^{N-1})$, which has Euclidean norm $\sqrt{N}$, the random variable $a(\zeta_M)$ has variance $V = \sigma^2 N$, where σ^2 is the variance of each coefficient of a . Hence, $a(\zeta_M)$ has variances $V_U = q^2 N/12$, $V_G = \sigma^2 N$, and $V_Z = \rho N$ when a is sampled from $U(R_q)$, $\text{DG}(\sigma^2)$, and $\text{ZO}(\rho)$, respectively. In particular, $a(\zeta_M)$ has variance $V_H = h$ when $a(X)$ is chosen from $\text{HWT}(h)$. Moreover, we can assume that $a(\zeta_M)$ is distributed approximately as a Gaussian random variable over the complex plane, since it is a sum of many independent and identically distributed random variables. All evaluations at roots of unity ζ_M^j share the same variance. Hence, we adopt 6σ as a high-probability bound on the canonical embedding norm of a when each coefficient has variance σ^2 . For the product of two independent random variables that are approximately Gaussian with variances σ_1^2 and σ_2^2 , we use a high-probability bound of $16\sigma_1\sigma_2$.

C.1 Lemma 1

LEMMA 1. Let $m(X) = \text{Ecd}(z; \Delta)$ and $c = \text{Enc}_{\text{pk}}(m(X))$ be a fresh ciphertext. Let $\epsilon_{\text{enc}} = m(X) - \text{Dec}_{\text{sk}}(c)$. Then, except with negligible probability, $\|\epsilon_{\text{enc}}\|_{\text{can}, \infty} \leq B_{\text{enc}} = 8\sqrt{2}\sigma N + 6\sigma\sqrt{N} + 16\sigma\sqrt{hN}$. Furthermore, decoding is robust given $\Delta > N + 2B_{\text{enc}}$. By construction, $B_{\text{refresh}} \gtrsim B_{\text{enc}}$.

PROOF. The encryption noise bound B_{enc} is computed via the following inequality:

$$\| \langle c, \text{sk} \rangle - m \bmod q_L \|_{\text{can}, \infty} = \| v \cdot e + e_0 + e_1 \cdot \text{sk} \|_{\text{can}, \infty}.$$

We apply the triangle inequality to obtain:

$$\leq \| v \cdot e \|_{\text{can}, \infty} + \| e_0 \|_{\text{can}, \infty} + \| e_1 \cdot \text{sk} \|_{\text{can}, \infty}.$$

Each term is bounded individually as follows:

$$\|v \cdot e\|_{\text{can},\infty} \leq 8\sqrt{2} \cdot \sigma N, \quad \|e_0\|_{\text{can},\infty} \leq 6\sigma\sqrt{N},$$

$$\|e_1 \cdot \text{sk}\|_{\text{can},\infty} \leq 16\sigma\sqrt{hN}.$$

Thus, the total encryption noise satisfies:

$$B_{\text{clean}} \leq 8\sqrt{2} \cdot \sigma N + 6\sigma\sqrt{N} + 16\sigma\sqrt{hN}.$$

□

C.2 Lemma 2

LEMMA 2. Let z be a plaintext vector and let Δ be the scaling factor. Define $\tilde{m} = \Delta(\pi^{-1}(z))$, $\hat{m} = \lfloor \tilde{m} \rfloor \in R$ with the rounding error $\epsilon_{\text{ecd}} = m - \hat{m}$. We have: $\|\epsilon_{\text{ecd}}\|_{\text{can},\infty} \leq B_{\text{ecd}} = \frac{\|\sigma_1\|_{\text{op}}}{2\Delta}$, where $\|\sigma_1\|_{\text{op}} \approx \sqrt{N}$.

PROOF. Each coefficient of $\tilde{m} = \Delta \pi^{-1}(z)$ is a real number. Rounding coordinate-wise yields $\hat{m} = \tilde{m} + \epsilon_R$ with $\epsilon_R := \tilde{m} - \hat{m} \in R$ and $\|\epsilon_R\|_{\infty} \leq \frac{1}{2}$. Because the coefficients were scaled by Δ , we have $\|\epsilon_R\|_{\infty} \leq \frac{1}{2\Delta}$. By definition $p(\epsilon_{\text{enc}}) = \epsilon_R$, so

$$\|p(\epsilon_{\text{enc}})\|_{\infty} \leq \frac{1}{2\Delta}.$$

Finally, applying the linear map σ_1 multiplies any vector norm by at most $\|\sigma_1\|_{\text{op}}$, hence

$$\|\sigma_1(p(\epsilon_{\text{enc}}))\|_{\infty} \leq \frac{\|\sigma_1\|_{\text{op}}}{2\Delta}.$$

□

C.3 Lemma 3

LEMMA 3. Let (c_i, Δ_i, B_i) encrypt $m_i \in R$ for $i \in \{1, 2\}$. Let $v_i := \|m_i\|_{\text{can},\infty}$ and set $v := \max(v_1, v_2)$. $\tilde{c} := \text{MULTCONST}(\frac{\Delta}{\Delta_1 \Delta_2}, c_{\text{mult}})$. Then, except with negligible probability, for the common fixed-scale case $\Delta_1 = \Delta_2 = \Delta$, we have $\|\text{Dec}(\tilde{c}) - m_1 m_2\|_{\text{can},\infty} \leq B_{\text{mult}}$ and

$$B_{\text{mult}} = \frac{v(B_2 + B_1) + B_1 B_2 + 16\sqrt{3}\sigma^2 N}{\Delta}.$$

PROOF. Write $\text{Dec}(c_i) = m_i + e_i$ so $\|e_i\|_{\text{can},\infty} \leq B_i$. Before relinearization,

$$(b_1 + a_1 s)(b_2 + a_2 s) = d_0 + d_1 s + d_2 s^2.$$

Adding $d_2 \cdot \text{evk}$ replaces $d_2 s^2$ by $d_2(s^2 + e_{\text{evk}})$, so the decrypted value of c_{mult} equals $(m_1 + e_1)(m_2 + e_2) + d_2 e_{\text{evk}}$. Hence, the pre-scaling error is $e_{\text{mult}} = m_1 e_2 + m_2 e_1 + e_1 e_2 + d_2 e_{\text{evk}}$. By submultiplicativity in the canonical embedding, $\|m_1 e_2\| \leq v_1 B_2$, $\|m_2 e_1\| \leq v_2 B_1$, $\|e_1 e_2\| \leq B_1 B_2$ and $\|d_2 e_{\text{evk}}\| \leq \|d_2\| \|e_{\text{evk}}\| = U_{\times} B_{\text{evk}}$. Finally, multiplying the ciphertext by $\Delta/(\Delta_1 \Delta_2)$ scales the error by the same factor. Using the assumption for concrete bounds as shown in the lemma statement. □

C.4 Lemma 4

LEMMA 4. Let c_1, c_2 be ciphertexts encrypting $m_1, m_2 \in R$ with decryption noise bounded by B_1 and B_2 of the same scale Δ . Let $c_{\text{add}} = c_1 + c_2$. Then: $\|\epsilon_{\text{add}}\|_{\text{can},\infty} \leq B_{\text{add}} = B_1 + B_2$.

PROOF. Homomorphic addition of ciphertexts is component-wise: $c_{\text{add}} = c_1 + c_2$. The decryption of c_{add} yields: $\text{Dec}(c_{\text{add}}) = m_1 + m_2 + (\epsilon_1 + \epsilon_2)$. Therefore:

$$\|\epsilon_{\text{add}}\|_{\infty} \leq \|\epsilon_1\|_{\infty} + \|\epsilon_2\|_{\infty} \leq B_1 + B_2.$$

Correctness holds as long as $\Delta > 2 \cdot \|\epsilon_{\text{add}}\|_{\infty}$. □

C.5 Lemma 5

LEMMA 5 (ADDITION AND MULTIPLICATION BY CONSTANT). Let (c, B_c) be a CKKS encryption of m . For a constant $a \in R$, $c_a := c + (a, 0)$, $c_m := a \cdot c$, where the ciphertexts (c_a, B_c) and $(c_m, \|a\|_{\text{can},\infty} \cdot B_c)$ are valid encryptions of $m + a$ and $a \cdot m$, respectively.

PROOF. By correctness of the proposed framework, the inner product of c and the secret key yields $m + e$ for some error polynomial e with $\|e\|_{\text{can},\infty} \leq B$. For the addition case, since the encoded constant a contributes no noise, the resulting error remains bounded by B . For multiplication, the ciphertext becomes $a \cdot (m + e) = am + ae$, which implies that the new error is bounded by $\|a\|_{\text{can},\infty} \cdot B$. □

C.6 Lemma 6

LEMMA 6 (POLYNOMIAL EVALUATION). Let $f(x) = \sum_{j=0}^d a_j x^j$ with real coefficients a_j , and let (c_0, B_0) be a valid ciphertext encrypting $m \in R$ with initial coefficient-error bound $B_0 = B_{\text{enc}}$. Fix a refresh threshold B^* . The extended Algorithm 2 outputs a ciphertext (c_f, B_f) such that $c_f = \text{Enc}(f(m))$ with $B_f \approx \min\{\sum_{j=0}^d |a_j| B_{\text{enc}}, B^*\}$.

PROOF. All norms are $\|\cdot\|_{\infty}$. One squaring (multiply, relinearize, exact rescale) of an encryption of m with error e gives

$$\text{Dec}(\text{Mult}(c, c))/\Delta - m^2 = me + em + e^2 + e_{\text{ks}},$$

so with $v = \|m\|_{\infty}$ and $C_{\text{ks}} = 16\sqrt{3}\sigma^2 N$,

$$B_{\text{sq}}(B) \leq \frac{2vB + B^2 + C_{\text{ks}}}{\Delta}.$$

The algorithm sets the new error to $\min\{B_{\text{sq}}(B), B^*\}$, and calls **Refresh** whenever needed to clamp it to B_{enc} . Hence after any powering chain (binary powering for m^j), the monomial ciphertext has error $\leq \min\{G(B_{\text{enc}}), B^*\} \leq B^*$, and if a refresh occurred then $\leq B_{\text{enc}}$.

Plaintext scaling by a_j multiplies the error by $|a_j|$. Summing monomials adds errors. Execute the final sum without refresh while the running bound stays $\leq B^*$; if it would exceed B^* , refresh once to B_{enc} and continue. Consequently,

$$B_f \leq \min\left\{\sum_{j=0}^d |a_j| B_{\text{enc}}, B^*\right\}.$$

All bounds hold except with negligible probability from subgaussian tails of encryption and EVK noise. □

C.7 Proof of Proposition 1

PROOF. Followed by simplifying the multiplication noise equations. □

C.8 Proof of Theorem 3

PROOF SKETCH. Since $f(x)$ is analytic, it equals its power series expansion in some radius of convergence around 0. We choose D such that the omitted tail $\sum_{j>D} a_j m^j$ is at most ϵ in magnitude (by an appropriate form of Taylor's remainder bound). The truncated polynomial $P_D(x) = \sum_{j=0}^D a_j x^j$ is then evaluated homomorphically as described. Each exponentiation x^j is performed by repeated squaring with refresh: after each multiplication, the noise would grow to roughly $(1+h)B^2$ (dominated by the quadratic term in B), so we refresh whenever the noise bound would exceed B^* . This ensures that each power x^j is obtained with noise $\approx B_{\text{enc}}$ (by Lemma 6). Multiplying by the plaintext coefficient a_j does not increase the noise. Summing the encrypted terms contributes additive noise up to $\sum_{j=0}^D |a_j| B_{\text{enc}}$. Finally, because we cap the noise at B^* via refresh, the output noise B_f is at most B^* . In summary,

$$B_f \approx \min \left\{ \sum_{j=0}^D |a_j| B_{\text{enc}}, B^* \right\},$$

and including the truncation error ϵ from ignoring high-degree terms, the total error is bounded by ϵ as required. $\square$

Note: Given Theorem 3 and Lemma 6, we compare the accuracy of polynomial evaluation under standard CKKS and our non-leveled CKKS variant by analyzing the resulting *relative error*. Let $f(x) = \sum_{j=0}^d a_j x^j$ be a polynomial with coefficients $a_j \in \mathbb{R}$. Recall the relative error of a ciphertext as $\beta := B_f/\Delta$, where B_f is the ciphertext's absolute error and Δ is the plaintext scaling factor.

In standard CKKS, the evaluated ciphertext is typically tracked as $(c', \ell - \lceil \log_2 d \rceil, M_f, \beta_d M_f)$, and its relative noise satisfies $\beta_d \leq d\beta_0 + (d-1)\beta^*$, where β_0 is the initial relative error and β^* is the per-level noise introduced by rescaling.

In our variant, the resulting ciphertext has absolute noise

$$B_f = \min \left\{ \sum_{j=0}^d |a_j| B_{\text{enc}}, B^* \right\},$$

and hence the relative error is

$$\beta_f = \min \left\{ \frac{\sum_{j=0}^d |a_j| B_{\text{enc}}}{\Delta}, \beta^* \right\}.$$

In the unsaturated regime $\sum_{j=0}^d |a_j| B_{\text{enc}} \leq B^*$, we compare relative errors by checking when

$$\frac{\sum_{j=0}^d |a_j| B_{\text{enc}}}{\Delta} < d\beta_0 + (d-1)\beta^*.$$

C.9 Proof of Theorem 4

PROOF. We use a sequence of games.

- Game G_0 (Real IND-CPA): This is the standard experiment for Π : **KeyGen** samples $s \leftarrow \text{HWT}(h)$, $a \leftarrow R_q$, $e \leftarrow \chi$, $b := -as + e$ and publishes $pk = (b, a)$. Encryptions use fresh $v \leftarrow \{0, 1\}^N$ and $e_0, e_1 \leftarrow \chi$: $(c_0, c_1) = (vb + m + e_0, va + e_1)$.
- Game G_1 : Identical to G_0 except that every occurrence of (va, vb) is replaced by a uniformly random $(u_1, u_0) \leftarrow R_q^2$ independently of v .
- Game G_2 : The challenger draws $(a^*, b^*) \leftarrow R_q^2$ uniformly and sets $pk = (b^*, a^*)$. Encryption algorithm is as in G_1 (outputting $(u_0 + m + e_0, u_1 + e_1)$).

Claim. G_0 and G_1 are perfectly indistinguishable: conditional on (e_0, e_1) , each term $(va + e_1, vb + e_0)$ is uniform over R_q^2 whether v is 0 or 1.

Claim. $G_1 \equiv G_2$ because the public key plays no role after the uniform pair (u_1, u_0) is used.

We build a reduction $\mathcal{B}$: given a sample (a^*, b^*) drawn from either $\mathcal{D}_0$ or $\mathcal{D}_1$ (RLWE Assumption), $\mathcal{B}$ plugs it in as the public key and simulates G_0 for $\mathcal{A}$ using *real* encryption randomness. If (a^*, b^*) is RLWE, the simulation is identical to G_0 . If uniform, it realizes G_2 . Thus

$$|\Pr[\mathcal{A} \text{ wins } G_0] - \Pr[\mathcal{A} \text{ wins } G_2]| = \text{Adv}_{\mathcal{B}}^{\text{RLWE}(h)}.$$

Combining the two claims yields

$$\begin{aligned} \text{Adv}_{\Pi}^{\text{ind-cpa}}(\mathcal{A}) &= |\Pr_{G_0}[b' = b] - \frac{1}{2}| \\ &= |\Pr_{G_2}[b' = b] - \frac{1}{2}| \leq \text{Adv}_{\mathcal{B}}^{\text{RLWE}(h)}. \end{aligned}$$

Hence Π is IND-CPA secure under the R-LWE Assumption. $\square$

C.10 Proof of Theorem 5

PROOF. Let $E : \{0, 1\}^k \rightarrow \{0, 1\}^n$ be the public, deterministic BCH encoder and D its inverse decoder. Define a wrapper adversary $\mathcal{B}$ that replaces every query m of $\mathcal{A}$ by $E(m)$ and forwards the resulting ciphertext unchanged; upon decryption it applies D . Because E is injective and public, the following holds:

$$\text{Adv}_{S^*}^{\text{ind-cpa}}(\mathcal{A}) \leq \text{Adv}_S^{\text{ind-cpa}}(\mathcal{B}),$$

which is identical to Theorem 4. $\square$